Taller Práctico

Visualización #4

Por: Dra. Gema Castillo

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
DEPARTAMENTO DE SISTEMAS DE INFORMACIÓN, CONTROL Y EVALUACIÓN DE RECURSOS INFORMÁTICOS

Actividad 4

Trabajo Individual.

A. Título de la Experiencia: Análisis: Cuatro Niveles para la Validación

B. Temas: Se desarrollan los conceptos fundamentales del Capítulo 4 de Visualization Analysis and Design (Munzner, 2014): los cuatro niveles anidados del diseño (situación de dominio, abstracción de tarea/datos, Idiom, algoritmo), las amenazas a la validez de cada nivel y los métodos de validación correspondientes.

C. Objetivo: Aplicar el modelo de cuatro niveles para diagnosticar decisiones de diseño de visualización y practicar validaciones básicas de idioma y de algoritmo mediante actividades en Excel y Python.

D. Metodología: Seguir las instrucciones de este taller.

E. Recursos: Conexión a internet, PC o laptop, Microsoft Excel y Python (Jupyter/Google Colab). Se sugiere leer el Capítulo 4 de Visualization Analysis and Design (Munzner, 2014) para mayor entendimiento.

F. Rúbrica:

Reporte individual con los puntos solicitados en este taller, según lo requerido. Cada actividad práctica equivale a 25 puntos. Total 100 puntos.

1. Desarrollo Conceptual

1.1. ¿Por qué validar?

Como se explicó en el Capítulo 1, el espacio de diseño de la visualización es enorme y la mayoría de los diseños posibles resultan inefectivos. Por ello, conviene pensar en cómo se validará una decisión de diseño desde el inicio del proceso, y no dejarlo como una idea de último momento. Este capítulo aporta un marco de cuatro niveles que ayuda a decidir qué método de validación es apropiado según en qué parte del proceso de diseño se encuentre el problema.

1.2. Los cuatro niveles anidados del diseño

El modelo divide el problema de diseño en cuatro niveles anidados: en el nivel más externo está la situación de dominio; le sigue la abstracción de tarea y de datos (el nivel qué–por qué explicado en los Capítulos 2 y 3); después viene el idiom de codificación visual e interacción (el nivel cómo, que se desarrolla en detalle a partir del Capítulo 5); y en el nivel más interno está el algoritmo, que instancia computacionalmente el idiom elegido. La salida de cada nivel es la entrada del siguiente nivel hacia adentro, por lo que una mala decisión en un nivel superior arrastra el problema a todos los niveles posteriores, incluso si estos se diseñan de forma impecable.

1.3. Nivel 1: Situación de dominio

En este nivel se describe un grupo de usuarios objetivo, su dominio de interés, sus preguntas y sus datos, usando el vocabulario propio de ese dominio. Los bloques de este nivel se identifican —no se diseñan— mediante entrevistas, observación o investigación cuidadosa del contexto de los usuarios reales. Una pregunta demasiado general no es útil como entrada para el siguiente nivel; se necesita el mismo nivel de detalle y especificidad que exigiría un verdadero levantamiento de requerimientos.

1.4. Nivel 2: Abstracción de tarea y de datos

En este nivel se traduce el lenguaje específico del dominio a un vocabulario genérico e independiente del dominio: los bloques de tarea se identifican (usando el vocabulario del Capítulo 3, como comparar, resumir o explorar), mientras que los bloques de datos se diseñan, ya que a menudo conviene transformar el conjunto de datos original en una forma distinta y más útil para el problema (usando las ideas de derivación de datos del Capítulo 3 y los tipos del Capítulo 2).

1.5. Nivel 3: Idioma de codificación visual e interacción

En este nivel se decide la forma concreta de crear y manipular la representación visual del bloque de datos elegido, guiada por la tarea abstracta identificada en el nivel anterior. Existen dos grandes conjuntos de decisiones: el idioma de codificación visual (qué ve el usuario en una imagen estática) y el idioma de interacción (cómo el usuario cambia lo que ve). Estos bloques se diseñan, y la decisión debe justificarse a partir de las capacidades perceptuales y cognitivas humanas, tema que se profundiza en los Capítulos 5 y 6.

1.6. Nivel 4: Algoritmo

El nivel más interno abarca las decisiones necesarias para crear un procedimiento detallado que permita a la computadora ejecutar de manera eficiente el Idioma elegido. Este nivel también se diseña, y el foco de las decisiones pasa de las capacidades humanas a las capacidades computacionales: velocidad de cómputo, uso de memoria y fidelidad del resultado respecto al idioma especificado.

1.7. Enfoques de ataque: de arriba hacia abajo y viceversa

Existen dos formas comunes de abordar el diseño. En el trabajo dirigido por el problema (problem-driven), se comienza en el nivel superior de situación de dominio y se desciende progresivamente por los niveles de abstracción, idioma y algoritmo; este es el enfoque típico de un estudio de diseño (design study) para un usuario real. En el trabajo dirigido por la técnica (technique-driven), se comienza directamente en uno de los dos niveles inferiores —idioma o algoritmo— con el objetivo de inventar un nuevo idioma que soporte mejor abstracciones ya existentes, o un nuevo algoritmo que soporte mejor idioms ya existentes.

1.8. Amenazas a la validez y métodos de validación

Cada uno de los cuatro niveles tiene una amenaza a la validez característica y requiere un método de validación distinto. En el nivel de situación de dominio, la amenaza es el problema equivocado (haber malentendido las necesidades reales de los usuarios), y se valida observando y entrevistando a los usuarios objetivo. En el nivel de abstracción, la amenaza es la abstracción equivocada (mostrarles algo que no responde a su verdadera necesidad), y se valida justificando la elección frente a alternativas y con retroalimentación de usuarios. En el nivel de idioma, la amenaza es un idioma inefectivo (la forma de mostrarlo no funciona perceptualmente), y se valida con análisis cualitativo de las imágenes resultantes, estudios de laboratorio que midan tiempo y errores, o pruebas de usabilidad. En el nivel de algoritmo, la amenaza es un algoritmo lento, y se valida con análisis de complejidad computacional y mediciones de tiempo y memoria del sistema implementado.

Es importante notar que la mayoría de las validaciones de los niveles externos (situación, abstracción) no son inmediatas: dependen de que los niveles internos (Idioma, algoritmo) ya estén implementados, porque solo así se puede comprobar en la práctica si la solución completa funciona con usuarios reales. Un mal resultado en una prueba de usuario puede deberse a un idioma mal diseñado, aunque la abstracción de tarea y de datos haya sido, en el fondo, correcta.

Actividades prácticas

Las siguientes actividades utilizan datos ya preparados. Junto con este taller se entregan: Datos_Capitulo4.xlsx (hojas "Promedio_Grupo_Lab" y "Notas_Completas" para Excel) y los archivos promedio_grupo_lab_capitulo4.csv y notas_completas_capitulo4.csv (para Python). Descárgalos y colócalos en la misma carpeta donde trabajes.

Actividad 1. Análisis de los cuatro niveles de un proyecto real [Herramienta: Excel]

Se te proporciona, en Datos_Capitulo4.xlsx, la hoja "Promedio_Grupo_Lab" con el promedio ficticio de nota por grupo y laboratorio del curso Bases de Datos II.

- =PROMEDIO.SI.CONJUNTO(rango_Nota; rango_Grupo; grupo; rango_Lab; laboratorio) → para verificar un promedio específico de la hoja.
- Abre la hoja "Promedio_Grupo_Lab" y, en una tabla aparte, documenta los cuatro niveles del modelo aplicados a este caso: Situación de dominio, Abstracción de tarea/datos, Idiom, Algoritmo.
- En una columna, describe brevemente la decisión que tomarías en cada nivel para atender la necesidad.
- En una segunda columna, indica cuál sería la amenaza a la validez característica de ese nivel si la decisión estuviera mal.
- En una tercera columna, indica qué método de validación usarías para comprobar que la decisión de ese nivel fue correcta.

Actividad 2. Validación de idioma mediante análisis de imagen y tiempo [Herramienta: Excel]

Se te proporciona, en el mismo archivo, la hoja "Promedio_Grupo_Lab" (15 filas: 3 grupos x 5 laboratorios) para comparar dos formas distintas de visualizar el mismo conjunto de datos.

- Tabla dinámica: Grupo en Filas, Laboratorio en Columnas, Promedio de Nota en Valores
- =PROMEDIO.SI.CONJUNTO(rango_Nota; rango_Grupo; "GS124"; rango_Lab; rango_Laboratorios) → cálculo manual de verificación
- =MIN(rango_Promedios_GS124) → identifica el laboratorio con más dificultad para ese grupo
- Con los datos de "Promedio_Grupo_Lab", crea dos gráficos distintos: un gráfico de barras agrupadas (Laboratorio en el eje X, una serie por Grupo) y un mapa de calor con formato condicional de escala de color (usa una tabla dinámica con Grupo en filas y Laboratorio en columnas).
- Cronometra, con un compañero, cuánto tarda en responder correctamente a la pregunta “¿en qué laboratorio tuvo más dificultad el Grupo GS124?” usando cada gráfico.
- Registra los tiempos y errores en una tabla comparativa.
- Concluye cuál idioma es más efectivo para esa tarea específica, siguiendo la lógica de validación por estudio de laboratorio del capítulo.

Actividad 3. Medición de tiempo y complejidad de un algoritmo en Python [Herramienta: Python]

Se te proporciona el archivo notas_completas_capitulo4.csv con 180 registros ficticios (36 estudiantes x 5 laboratorios) del curso Bases de Datos II para practicar la validación del nivel de algoritmo con un caso real, y además generarás datos sintéticos más grandes para medir el efecto de la escala.

- df.groupby('Grupo')['Nota'].mean()
- import time; inicio = time.time(); ...; duracion = time.time() - inicio

- Carga notas_completas_capitulo4.csv con pandas y calcula el promedio de nota por grupo usando dataframe.groupby('Grupo')['Nota'].mean(); mide el tiempo con la librería time.
- Genera ahora un DataFrame sintético mucho más grande (por ejemplo, 200,000 filas simuladas) con pandas y numpy, replicando la misma estructura de columnas.
- Implementa la misma agregación (promedio por grupo) primero con un ciclo for manual en Python puro sobre el DataFrame grande, y mide su tiempo; luego con dataframe.groupby().mean(), y mide su tiempo.
- Compara los tiempos obtenidos con el dataset real (180 filas) y con el sintético (200,000 filas), y redacta una conclusión relacionándola con la idea de "algoritmo equivocado" y su validación mediante análisis de complejidad y medición de tiempo del sistema.

Actividad 4. Prototipo rápido para validar la abstracción antes de programar [Herramienta: Python]

Se te proporciona nuevamente el archivo notas_completas_capitulo4.csv para simular la práctica de "validación temprana" (prototipos de baja fidelidad) mencionada en el capítulo, construyendo primero un boceto simple antes de invertir tiempo en un idioma más elaborado.

- df.groupby(['Grupo','Laboratorio'])['Nota'].mean().unstack() → tabla de promedios lista para graficar
- matplotlib.pyplot.plot(...) → prototipo de línea simple
- Define, en un comentario de tu script, la situación de dominio y la abstracción de tarea/datos para el caso (comparar el desempeño de tres grupos en cinco laboratorios).
- Carga notas_completas_capitulo4.csv y construye un primer prototipo muy simple con matplotlib (por ejemplo, un gráfico de líneas básico sin formato, con el promedio por grupo y laboratorio) que solo busque confirmar si la abstracción elegida tiene sentido.
- Pide a un compañero que interprete el prototipo y responda si logra identificar qué grupo tiene más dificultad, sin explicaciones adicionales de tu parte.
- Con base en esa retroalimentación, decide si la abstracción de datos/tarea necesita ajustarse antes de invertir tiempo en un idioma más elaborado (por ejemplo, un mapa de calor con seaborn).

Bibliografía

Munzner, T. (2014). Visualization Analysis and Design. Boca Raton, FL: CRC Press, A K Peters Visualization Series.